

Structured Query Language - SQL



SQL

- DDL (*Data Definition Language*): linguagem de definição de dados;
- DML (*Data Manipulation Language*): linguagem de manipulação de dados;
- DQL (*Data Query Language*): linguagem de consulta de dados;
- DCL (*Data Control Language*): Linguagem de Controle de Dados;
- DTL (*Data Transaction Language*): Linguagem de Transação de Dados.

SQL

DDL

- CREATE
- DROP
- ALTER
- TRUNCATE

DML

- INSERT
- UPDATE
- DELETE

DCL

- GRANT
- REVOKE

TCL

- COMMIT
- ROLLBACK
- SAVEPOINT

DQL

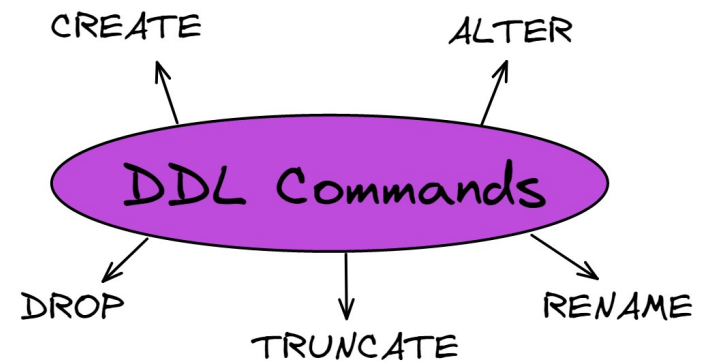
- SELECT

Structured Query Language (SQL)

DDL

■ Comandos

- CREATE
- DROP
- ALTER
- TRUNCATE



DDL – CREATE

```
CREATE TABLE nome_tabela  
  ( coluna      tipo      [constraint coluna],  
    ...  
    ...  
    ...  
  [constraint de tabela] );
```

DDL – CREATE – Tipos de Dados

Categoria	Tipo	Descrição	Exemplo de uso
Numéricos	INTEGER ou INT	Número inteiro de 4 bytes	idade INT
	SMALLINT	Número inteiro de 2 bytes	quantidade SMALLINT
	BIGINT	Número inteiro de 8 bytes	populacao BIGINT
	NUMERIC(p,s)	Número com precisão e escala definidas (exato)	salario NUMERIC(10,2)
	REAL	Número de ponto flutuante (4 bytes)	temperatura REAL
	DOUBLE PRECISION	Número de ponto flutuante (8 bytes)	media DOUBLE PRECISION

DDL – CREATE – Tipos de Dados

Texto	<code>VARCHAR(n)</code>	Cadeia de texto com tamanho máximo	<code>nome VARCHAR(50)</code>
	<code>CHAR(n)</code>	Texto de tamanho fixo	<code>sigla CHAR(2)</code>
	<code>TEXT</code>	Texto de tamanho ilimitado	<code>descricao TEXT</code>
Booleano	<code>BOOLEAN</code>	Verdadeiro ou falso (<code>true</code> / <code>false</code>)	<code>ativo BOOLEAN</code>
Data/Hora	<code>DATE</code>	Data (ano, mês, dia)	<code>data_nascimento DATE</code>
	<code>TIME</code>	Hora (sem data)	<code>horario_abertura TIME</code>
	<code>TIMESTAMP</code>	Data e hora	<code>criado_em TIMESTAMP</code>
	<code>INTERVAL</code>	Período de tempo (duração)	<code>prazo INTERVAL</code>
Monetário	<code>MONEY</code>	Valor monetário com formatação local	<code>preco MONEY</code>
Binário	<code>BYTEA</code>	Armazena dados binários (ex: arquivos, imagens)	<code>imagem BYTEA</code>

DDL – CREATE – Tipos de Dados

UUID	UUID	Identificador universal único	id UUID
JSON/XML	JSON	Texto JSON sem validação interna	dados JSON
	JSONB	Texto JSON binário (mais eficiente e indexável)	config JSONB
	XML	Documento XML	conteudo XML
Array	tipo[]	Vetores de qualquer tipo (ex: INT[] , TEXT[])	tags TEXT[]
Enumerado	ENUM	Conjunto de valores definidos pelo usuário	estado ENUM ('novo', 'usado')
Geoespacial	POINT , LINE , CIRCLE , etc.	Tipos geométricos (usado com PostGIS)	localizacao POINT

DDL – CREATE

```
CREATE TABLE pessoa (  
    codigo INTEGER PRIMARY KEY,  
    fone VARCHAR(15) NOT NULL,  
    nome VARCHAR(35) NOT NULL );
```

```
CREATE TABLE pessoa (  
    codigo INTEGER,  
    fone VARCHAR(15) NOT NULL,  
    nome VARCHAR(35) NOT NULL,  
    PRIMARY KEY (codigo) );
```

DDL – CREATE - Constraints

- NOT NULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY
- CHECK

Constraints

Primary Key

Foreign Key

Unique Key



Banco de Dados

Default

Check

Not Null

DDL – CREATE - Constraints

```
CREATE TABLE TB_PESSOA(  
    codigo    INTEGER      PRIMARY KEY,  
    fone      VARCHAR(15) NOT NULL,  
    nome      VARCHAR(35) NOT NULL,  
    cpf       CHAR(13) UNIQUE,  
    casado    INTEGER(7)  
    REFERENCES TB_PESSOA(codigo),  
    sexo      char(1) check (sexo IN ('F','M')) );
```

DDL – CREATE - Constraints

```
CREATE TABLE ESPECIALIDADE (  
    id      int AUTO_INCREMENT,  
    nome varchar(20) NOT NULL,  
    PRIMARY KEY (id)  
);
```

```
CREATE TABLE ESPECIALIDADE (  
    id      serial,  
    nome  character varying NOT NULL,  
    PRIMARY KEY (id)  
);
```

DDL – CREATE - Constraints

INSERT INTO ESPECIALIDADE (nome) VALUES ('OFTALMOLOGISTA');

INSERT INTO ESPECIALIDADE (nome) VALUES ('CARDIOLOGISTA');

id	nome
1	OFTALMOLOGISTA
2	CARDIOLOGISTA

DDL – DROP

- Elimina uma tabela;
- Todos os dados e estruturas da tabela são excluídos;
- Todas as transações pendentes serão efetivadas (commit);
- Todos índices serão eliminados.

```
DROP TABLE <nome da tabela>  
[CASCADE CONSTRAINTS];
```

DDL – ALTER

- Altera estrutura e constraints de uma tabela.
- O comando ALTER TABLE:
 - Adicionar e alterar colunas;
 - Incluir ou remover constraints;
 - Habilitar e desabilitar constraints.

ALTER TABLE distribuidores ADD COLUMN endereco varchar(30);

ALTER TABLE distribuidores DROP COLUMN endereco;

**ALTER TABLE distribuidores
ALTER COLUMN endereco TYPE varchar(80),
ALTER COLUMN nome TYPE varchar(100);**

DDL – ALTER

ALTER TABLE distribuidores RENAME COLUMN endereco TO cidade;

ALTER TABLE distribuidores RENAME TO fornecedores;

ALTER TABLE distribuidores ALTER COLUMN logradouro SET NOT NULL;

ALTER TABLE distribuidores ALTER COLUMN logradouro DROP NOT NULL;

**ALTER TABLE distribuidores ADD CONSTRAINT fk_endereco
FOREIGN KEY (endereco) REFERENCES enderecos (endereco);**

ALTER TABLE distribuidores ADD PRIMARY KEY (id_dist);

DDL – TRUNCATE

- Remove todas as linhas de uma tabela;
- Libera o espaço de armazenamento utilizado por esta tabela;
- Semelhante a instrução DELETE sem condição.

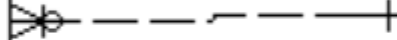
```
TRUNCATE TABLE <nome da tabela>;
```

Vamos Praticar!



veiculos	
placa: CHAR(8) NOT NULL [PK]	
chassi: VARCHAR(18) NOT NULL	
ano: INTEGER NOT NULL	
cor: VARCHAR(100) NOT NULL	
idcategoria: INTEGER NOT NULL [FK]	

categoria	
idcategoria: INTEGER NOT NULL [PK]	
nome: VARCHAR(200) NOT NULL	
descricao: VARCHAR(500)	



DML

- INSERT
- UPDATE
- DELETE



DML - INSERT

```
INSERT INTO tabela [colunas]  
VALUES (valores);
```

- Apenas uma linha é inserida por vez com esta sintaxe.

DML - INSERT

```
departamento  
(id      integer PRIMARY KEY,  
 nome    varchar(30)  NOT NULL,  
 regiao   integer REFERENCES regiao(id));
```

```
INSERT INTO departamento  
VALUES (1, 'VENDA', 1);
```

- Lista os valores na ordem das colunas na tabela;
- Coloque valores de caracter e de data entre apas simples.

DML - INSERT

```
INSERT INTO departamento (id,regiao)  
VALUES (1, 1);
```

```
INSERT INTO departamento  
VALUES (1, 'VENDA', NULL);
```

```
INSERT INTO departamento  
VALUES (1, 'VENDA', '');
```

DML - INSERT

- Inserindo várias linhas de uma vez só.

```
CREATE TABLE contacts (  
    id SERIAL PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(384) NOT NULL UNIQUE  
);
```

```
INSERT INTO contacts (first_name, last_name, email)  
VALUES  
    ('John', 'Doe', 'john.doe@example.com'),  
    ('Jane', 'Smith', 'jane.smith@example.com'),  
    ('Bob', 'Johnson', 'bob.johnson@example.com');
```

DML - UPDATE

```
UPDATE tabela  
SET   coluna = valor [,coluna=valor]  
[WHERE condição];
```

DML - UPDATE

```
UPDATE empregado  
SET    NumDep = 7  
WHERE  matrEmp='12345';
```

```
UPDATE  empregado  
SET     NumDep = 7, fone='222-3333'  
WHERE   matrEmp='12345';
```

DML - DELETE

```
DELETE [FROM] table  
[WHERE      condição];
```

DML - DELETE

- Remove todos os registros da tabela empregado:

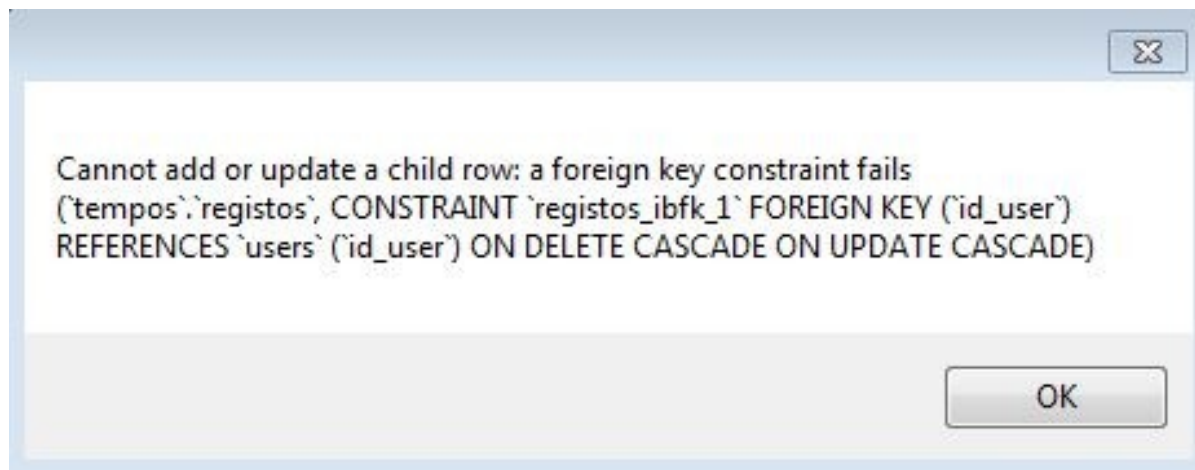
```
DELETE * FROM empregado;
```

- Remove apenas um registro da tabela empregado:

```
DELETE * FROM empregado WHERE  
matrEmp='12345';
```

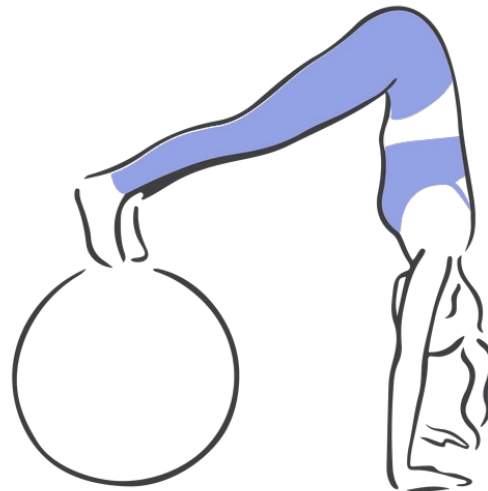
DML – DELETE - Erro de Constraint de Integridade

- Se você tentar excluir uma linha que contém uma chave primária usada como chave estrangeira em outra tabela, ocorrerá um erro de constraint de integridade.



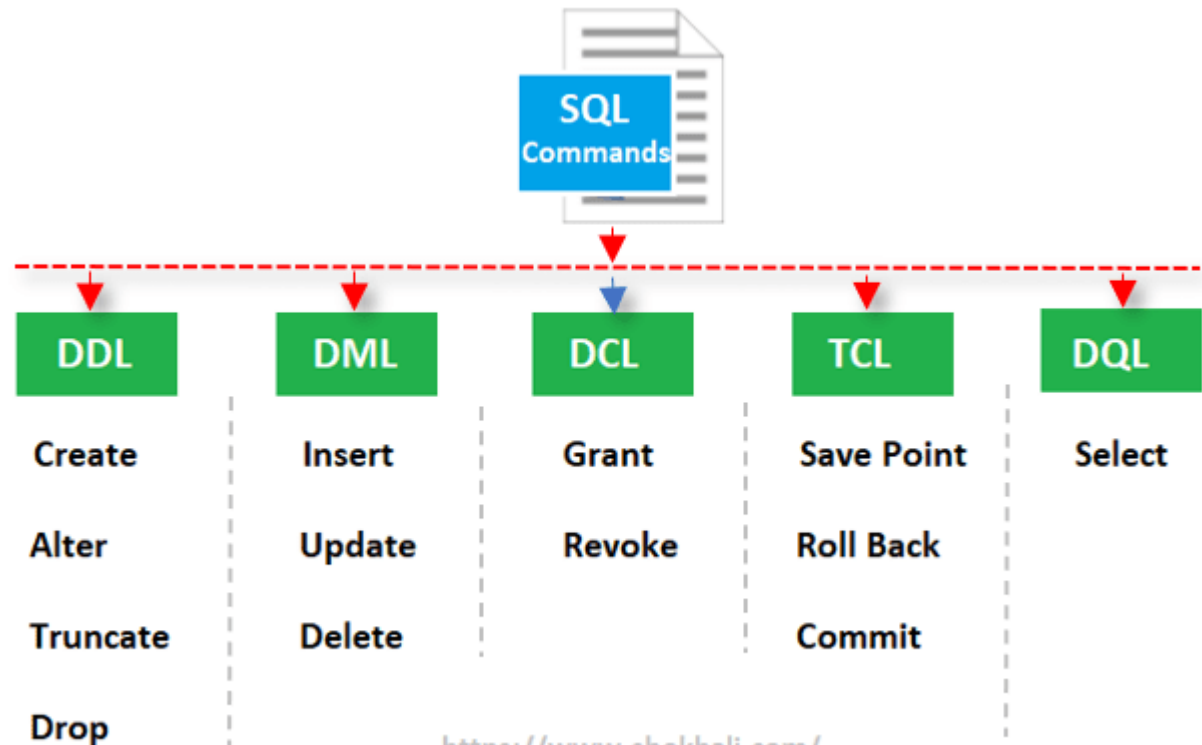
Vamos Praticar!

- Insira 5 registros em todas as tabelas do banco de dados veículos;
- Atualize 2 registros em todas as tabelas do banco de dados veículos;
- Apague 1 registro em todas as tabelas do banco de dados veículos.



DQL

■ SELECT



DQL - SELECT

Executa uma consulta no Banco de Dados

```
SELECT [DISTINCT] (*, coluna [alias])  
FROM tabela  
[WHERE condição]  
[ORDER BY {coluna, exp} [ASC|DESC]];
```

DISTINCT valores distintos (sem duplicação)

* todas as colunas da tabela

Alias apelido para a coluna na exibição dos dados

DQL - SELECT

Seleciona todos os dados de todas as colunas da tabela de Departamento:

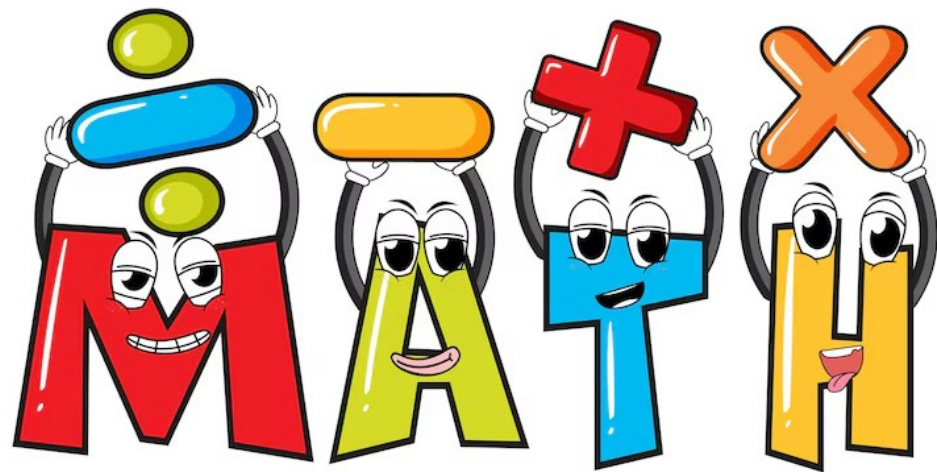
```
SELECT * FROM Departamento;
```

Seleciona todos os dados da coluna nome na tabela de Departamento:

```
SELECT nome FROM Departamento;
```

DQL – SELECT – Expressões Aritméticas

- Adição (+)
- Subtração (-)
- Multiplicação (*)
- Divisão (/)



DQL – SELECT – Expressões Aritméticas

Cálculo do salário anual dos empregados:

```
SELECT nome, salario*12 "Salario Anual"  
FROM Empregado;
```

Nome	Salario Anual
Maria	10000
Jose	5000
João	30000

DQL – SELECT – Expressões Aritméticas

Cálculo do salário anual dos empregados:

```
SELECT nome, salario*12 + salario + salario/3  
as "Salario Anual" FROM Empregado;
```

Nome	Salario Anual
Maria	10000
Jose	5000
João	30000

DQL – SELECT – Concatenação de Colunas

O operador de concatenação é || (duas barras verticais):

```
SELECT Prinome || UltimoNome “Nome do  
empregado” FROM Empregado;
```

Nome do Empregado
MariaSilva
JoseCosta
JoãoTornado

DQL – SELECT – DISTINCT

Seleciona todos os cargos da tabela de Empregado:

```
SELECT cargo FROM Empregado;
```

Cargo
Programador
Programador
Analista
Analista

DQL – SELECT – DISTINCT

Selecciona os diferentes tipos de cargos dos empregados:

```
SELECT DISTINCT cargo FROM Empregado;
```

Cargo
Programador
Analista

DQL – SELECT – ORDER BY

Seleciona os diferentes tipos de cargos dos empregados em ordem ascendente:

```
SELECT DISTINCT cargo FROM Empregado  
ORDER BY cargo;
```

```
Cargo  
Analista  
Programador
```

DQL – SELECT – ORDER BY

Seleciona os diferentes tipos de cargos dos empregados em ordem decrescente:

```
SELECT DISTINCT cargo FROM Empregado  
ORDER BY 1 DESC;
```

Cargo
Programador
Analista

DQL – SELECT – ORDER BY

Cálculo do salário anual dos empregados:

```
SELECT nome, salario*12 "Salario Anual"  
FROM Empregado ORDER BY 2;
```

Nome	Salario Anual
Jose	5000
Maria	10000
João	30000

DQL – SELECT – ORDER BY

Seleciona os diferentes tipos de cargos dos empregados em ordem crescente por departamento e ordem decrescente por salário:

```
SELECT nome, departamento, salarioa  
FROM Empregado ORDER BY departamento,  
salario DESC;
```

Nome	Departamento	Salario
Maria	10	1450
Jose	31	1400
Joao	31	750
Fabio	33	1600
Marcos	33	800

DQL – SELECT – WHERE

- Seleciona registro com uma determinada condição;
- Strings e datas devem estar entre aspas simples;
- Número não deve estar entre aspas simples;
- Valores de caracter podem ser **case-sensitive**.

DQL – SELECT – WHERE

Seleciona os nomes dos empregados que tem o cargo de Analista:

```
SELECT nome FROM Empregado WHERE  
cargo = 'Analista';
```

Nome
João

DQL – SELECT – Operadores de Comparação

Operador	Descrição	Exemplo
=	Igual a	WHERE idade = 18
<> ou !=	Diferente de	WHERE nome <> 'João'
>	Maior que	WHERE salario > 2000
<	Menor que	WHERE idade < 60
>=	Maior ou igual a	WHERE nota >= 7.0
<=	Menor ou igual a	WHERE ano <= 2023
BETWEEN	Dentro de um intervalo (inclusive)	WHERE idade BETWEEN 18 AND 30
IN	Dentro de um conjunto de valores	WHERE curso IN ('TI', 'ADM')
NOT IN	Fora de um conjunto de valores	WHERE cidade NOT IN ('SP', 'RJ')

DQL – SELECT – Operadores de Comparação

LIKE

Combinação de padrão textual (sensível a maiúsculas)

WHERE nome LIKE 'A%'

NOT LIKE

Não corresponde ao padrão textual (sensível a maiúsculas)

WHERE nome NOT LIKE '%Silva%'

ILIKE

LIKE sem diferenciar maiúsculas/minúsculas

WHERE nome ILIKE 'jo%'

IS NULL

Verifica se o valor é nulo

WHERE data_fim IS NULL

IS NOT NULL

Verifica se o valor **não** é nulo

WHERE telefone IS NOT NULL

DQL – SELECT – Operadores Lógicos

Operador	Descrição	Exemplo
AND	Todas as condições devem ser verdadeiras	<code>WHERE idade > 18 AND cidade = 'Recife'</code>
OR	Pelo menos uma condição deve ser verdadeira	<code>WHERE estado = 'SP' OR estado = 'RJ'</code>
NOT	Nega uma condição	<code>WHERE NOT ativo</code> ou <code>WHERE NOT nome LIKE 'A%'</code>

DQL – SELECT – WHERE

Selecione o nome e o salário dos empregados que recebem mais de 7.000 ordenados pelo salário:

```
SELECT      nome, salario
FROM        Empregado
WHERE       salario > 7000
ORDER BY    2;
```

Nome	Salario
Maria	10000
João	30000

DQL – SELECT – WHERE

SELECT	nome, salario
FROM	Empregado
WHERE	salario = 10000;

Nome	Salario
Maria	10000

DQL – SELECT – WHERE

SELECT	nome, salario
FROM	Empregado
WHERE	salario > 7000
AND	nome = 'Maria';

Nome	Salario
Maria	10000

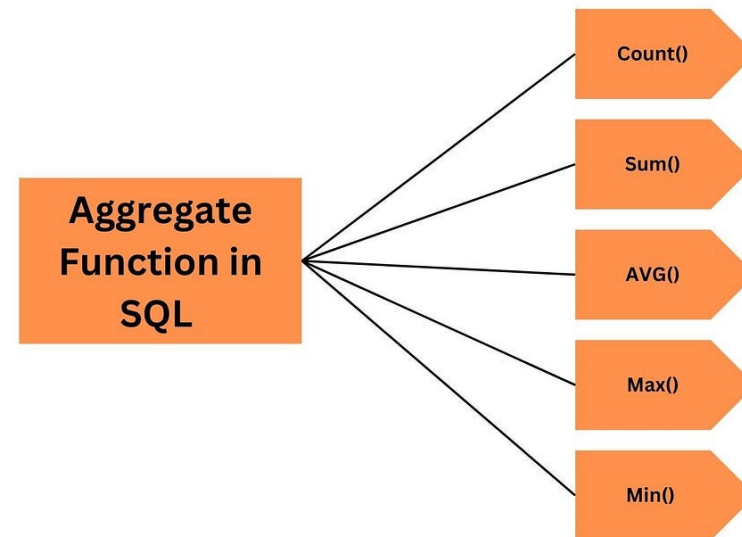
DQL – SELECT – WHERE

```
SELECT      nome, salario  
FROM        Empregado  
WHERE       salario > 7000  
OR          nome = 'Maria';
```

Nome	Salario
Maria	10000
João	30000

DQL – SELECT – Funções de Agregação

- AVG() – média;
- COUNT() – contador;
- MAX() – máximo;
- MIN() – mínimo;
- SUM() – mínimo.



DQL – SELECT – GROUP BY

- É usado para agrupar resultados de uma consulta com base nos valores de uma ou mais colunas, geralmente em conjunto com funções de agregação;
- Inclui ou exclui registros de grupos usando a clausura HAVING.



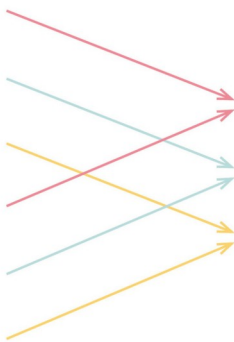
DQL – SELECT – GROUP BY

```
SELECT coluna, group_function  
FROM tabela  
[WHERE condição]  
[GROUP BY expressão]  
[HAVING condição do grupo]  
[ORDER BY colunas]
```

DQL – SELECT – GROUP BY

SELECT Nome, SUM(Valor) AS Total FROM Comissao
GROUP BY Nome;

SELECT Nome, SUM(Valor) AS Total FROM Comissao
GROUP BY Nome HAVING SUM(Valor) > 1000;



The diagram illustrates the process of grouping data. On the left is a source table with 6 rows. On the right is a result table with 3 rows. Colored arrows show the mapping: red arrows from 'book 1' and 'book 4' to 'adventure' in the result table; light blue arrows from 'book 2' and 'book 5' to 'fantasy'; and yellow arrows from 'book 3' and 'book 6' to 'romance'.

title	genre	qty
book 1	adventure	4
book 2	fantasy	5
book 3	romance	2
book 4	adventure	3
book 5	fantasy	3
book 6	romance	1

genre	total
adventure	7
fantasy	8
romance	3

DQL – SELECT – GROUP BY

Verifique qual é o maior, o menor, a soma e a média salarial da tabela de empregado:

```
SELECT  AVG(salario) "Média Salarial",  
        SUM(salario) "Soma dos salários"  
        MIN(salario) "Menor Salario"  
        MAX(salario) "Maior Salário"  
FROM    empregado;
```

DQL – SELECT – GROUP BY

Quanto departamento existem na tabela de departamento ?

```
SELECT COUNT(*)  
FROM departamento;
```

DQL – SELECT – GROUP BY

Quantos empregados do sexo M tem na tabela de empregado ?

```
SELECT count(*)  
FROM empregado  
WHERE sexo = 'M';
```

DQL – SELECT – GROUP BY

Qual a média salarial de cada cargo da tabela de empregado ?

```
SELECT cargo, AVG(salario) "Média Salarial"  
FROM empregado GROUP BY cargo;
```

DQL – SELECT – Produto Cartesiano

SELECT * FROM tabela1, tabela2;

Matr	Nome	Sexo	Dep
111	PEDRO	M	VE
222	MARIA	F	EN
123	CLAUDIA	F	VE

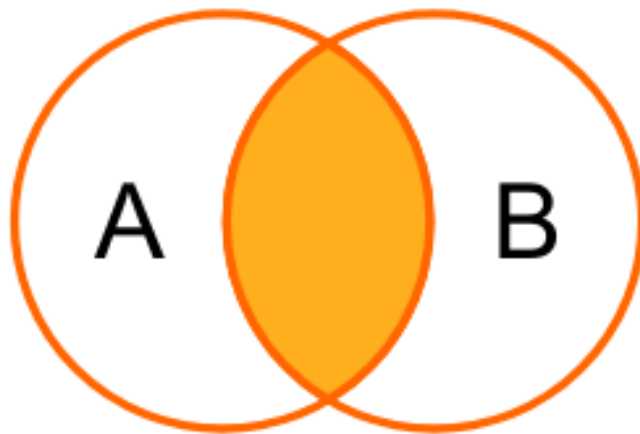
CodDep	NomeDep
VE	VENDAS
EN	ENGEN.

SELECT * FROM empregado, departamento;

	Nome	Sexo	Dep	CodDep	NomeDep
111	PEDRO	M	VE	VE	VENDAS
111	PEDRO	M	VE	EN	ENGEN.
222	MARIA	F	EN	EN	ENGEN.
222	MARIA	F	EN	VE	VENDAS
123	CLAUDIA	F	VE	VE	VENDAS
123	CLAUDIA	F	VE	EN	ENGEN.

DQL – SELECT – JOIN

```
SELECT tabela.coluna, tabela.coluna  
FROM tabela1, tabela2  
WHERE tabela1.coluna1 = tabela2.coluna2;
```



DQL – SELECT – JOIN

Matr	Nome	Sexo	Dep
111	PEDRO	M	VE
222	MARIA	F	EN
123	CLAUDIA	F	VE

CodDep	NomeDep
VE	VENDAS
EN	ENGEN.

**SELECT * FROM empregado, departamento
WHERE empregado.Dep = Departamento.codDep;**

Matr	Nome	Sexo	Dep	CodDep	NomeDep
111	PEDRO	M	VE	VE	VENDAS
222	MARIA	F	EN	EN	ENGEN.
123	CLAUDIA	F	VE	VE	VENDAS

DQL – SELECT – JOIN

Liste o nome do empregado e o nome do departamento onde ele trabalha:

```
SELECT empregado.nome, departamento.NomeDep  
FROM empregado, departamento  
WHERE empregado.Dep = departamento.codDep;
```

Nome	NomeDep
PEDRO	VENDAS
MARIA	ENGEN.
CLAUDIA	VENDAS

DQL – SELECT – JOIN

Liste o nome do empregado e o nome do departamento onde ele trabalha. Selecione apenas os empregados do sexo M:

```
SELECT empregado.nome, departamento.NomeDep  
FROM empregado, departamento  
WHERE empregado.Dep = departamento.codDep  
AND empregado.sexo = 'M';
```

Nome	NomeDep
PEDRO	VENDAS

DQL – SELECT – JOIN

```
SELECT * FROM table1, table2;
```

```
SELECT * FROM  
    table1 INNER JOIN table2 ON table1.id=table2.id;
```

```
SELECT * FROM  
    table1 LEFT JOIN table2 ON table1.id=table2.id;
```

```
SELECT * FROM  
    table1 RIGHT JOIN table2 ON table1.id=table2.id;
```

DQL – SELECT – JOIN

Cientes

id	nome
1	Ana
2	Bruno
3	Carla

Pedidos

id	cliente_id	valor
1	1	100.00
2	2	250.00
3	1	150.00
4	NULL	90.00

DQL – SELECT – JOIN

```
SELECT clientes.nome, pedidos.valor  
FROM clientes INNER JOIN pedidos  
ON clientes.id = pedidos.cliente_id;
```

nome	valor
Ana	100.00
Bruno	250.00
Ana	150.00

DQL – SELECT – JOIN

```
SELECT clientes.nome, pedidos.valor  
FROM clientes RIGHT JOIN pedidos  
ON clientes.id = pedidos.cliente_id;
```

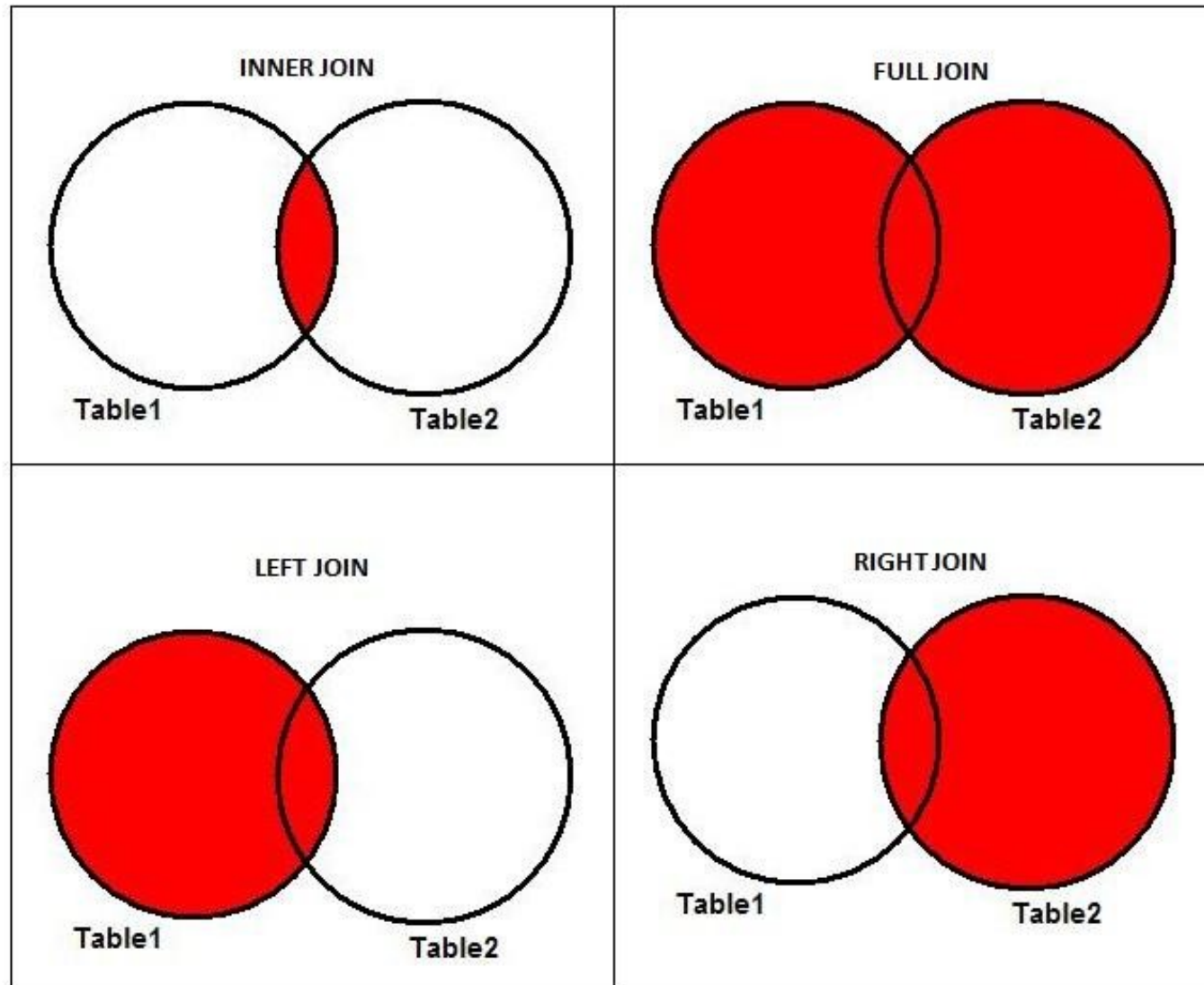
nome	valor
Ana	100.00
Bruno	250.00
Ana	150.00
NULL	90.00

DQL – SELECT – JOIN

```
SELECT clientes.nome, pedidos.valor  
FROM clientes LEFT JOIN pedidos  
ON clientes.id = pedidos.cliente_id;
```

nome	valor
Ana	100.00
Ana	150.00
Bruno	250.00
Carla	NULL

DQL – SELECT – JOIN



DQL – SELECT – UNION

```
SELECT id, msg FROM Mensagem  
UNION  
SELECT codigo, localidade FROM Postal;
```

- Em um UNION, o nome das colunas apresentado no Resultado é o nome das colunas selecionadas na primeira Instrução SELECT;
- Por default, uma UNION remove sempre linhas duplicadas;
- Se, em vez de UNION, utilizarmos o operador UNION ALL, as duplicatas não serão retiradas da seleção.

DQL – SELECT – UNION

clientes_brasil	id	nome
	1	Ana
	2	Bruno

clientes_argentina	id	nome
	1	Carla
	2	Bruno

DQL – SELECT – UNION

```
SELECT nome FROM clientes_brasil  
UNION  
SELECT nome FROM clientes_argentina;
```

nome

Ana

Bruno

Carla

DQL – SELECT – UNION

```
SELECT nome FROM clientes_brasil  
UNION ALL  
SELECT nome FROM clientes_argentina;
```

nome

Ana

Bruno

Carla

Bruno

DQL – SELECT – Subquery

- Uma subquery é uma consulta SQL interna (aninhada) usada dentro de outra consulta SELECT, INSERT, UPDATE ou DELETE.

Tipo	Exemplo comum
No WHERE	Filtrar com base em outra consulta
No FROM	Subquery como tabela derivada (tabela virtual)
No SELECT	Subquery como coluna
Com operadores IN, EXISTS, =	Para testes de condição com subconsulta

DQL – SELECT – Subquery

```
SELECT nome FROM clientes  
WHERE id IN ( SELECT cliente_id  
FROM pedidos);
```

Retorna Ana e Bruno, pois são clientes com pedidos.

```
SELECT nome, (SELECT COUNT(*) FROM  
pedidos WHERE pedidos.cliente_id = clientes.id)  
AS total_pedidos FROM clientes;
```

Mostra todos os clientes com a quantidade de pedidos que cada um fez.

DQL – SELECT – Subquery

```
SELECT media.valor_medio  
FROM (SELECT AVG(valor) AS  
valor_medio FROM pedidos) AS media;
```

Retorna a média de valor dos pedidos.

```
SELECT nome FROM clientes WHERE id IN  
( SELECT cliente_id FROM pedidos GROUP BY  
cliente_id HAVING SUM(valor) > 200);
```

Seleciona os nomes dos clientes cujo valor total de pedidos é maior que 200.

Vamos Praticar!

Faça as seguintes consultas no banco:

- a) Liste todos os veículos cadastrados;
- b) Liste todas as categorias cadastradas ordenadas pelo nome;
- c) Liste todos os veículos da cor preta;
- d) Liste todos os veículos da cor vermelha e do ano de 2013 ordenados pelo ano;
- e) Liste todos os veículos que possuem placa que começa com a letra R e termina com o número 1;
- f) Liste quantos veículos estão cadastrados nessa base de dados;
- g) Liste a placa de todos os veículos e os nomes de suas respectivas categorias;
- h) Liste a placa e o nome da categoria dos veículos do ano de 2010;
- i) Liste o nome das categorias e a quantidade de veículos de cada categoria;
- j) Listar as placas e cores dos veículos que estão associados à categoria "SUV";
- k) Listar os nomes das categorias que possuem mais de 2 veículos cadastrados.

FIM

